

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

"ConnectHub: A Multi-Client LAN Chat And File Sharing Application"

Submitted by

Prabesh BC (080BCT054)

Saroj Rawal (080BCT076)

Subesh Yadav (080BCT084)

A

PROJECT PROPOSAL

SUBMITTED TO THE

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

JULY, 2026

Abstract

ConnectHub is a multi-client LAN chat and file-sharing application developed in plain C, built entirely on standard POSIX system calls, sockets, pthreads, and select(). The system is designed to implement real-time network communication from first principles, without relying on any external frameworks or third-party libraries.

The server is capable of handling multiple client connections simultaneously through multi-threading, with mutexes used to protect shared resources such as the connected-user list and room membership data. The client is a lightweight terminal-based program that uses select() to monitor keyboard input and incoming network data at the same time, ensuring it never blocks on either source. Alongside public and private messaging, the system also allows users to share files with one another through the server.

All communication takes place over a custom, lightweight text protocol designed specifically for this project, using simple newline-separated messages. The implementation avoids GUI libraries and third-party dependencies entirely, relying only on C and standard system-level tools. The end result is a functional, well-tested chat and file-sharing application that reflects a solid, practical understanding of core systems programming and networking principles.

Table of Contents

Abstract	i
1. Introduction	1
2. Problem Statement	2
3. Objectives	3
3.1. Main Objective	3
3.2. Specific Objectives	3
4. Literature Review	3
4.1. Review of Related Theory	4
4.2. Review of Related Works.....	4
4.3. Gap Analysis.....	5
5. Proposed Methodology	6
5.1. Project Approach	6
5.2. System Architecture & Design Modules	6
5.3. Technology Stack	6
5.4. Application Protocol Design	7
5.5. Development Methodology	7
5.6. Concurrency & Data Handling Strategy.....	7
5.7. Testing & Validation Plan	7
5.8. Deployment Plan	8
5.9. Evaluation Metrics.....	8
6. Time Schedule	9
7. References	10

1. Introduction

Chat apps look simple from the outside, but underneath they rely on some of the most fundamental ideas in systems programming: sockets, concurrency, and a clear protocol for who sends what to whom [1]. Building one from scratch in C, without any framework doing the heavy lifting, is one of the best ways to actually understand how process/thread handling, synchronization, and network I/O work together [2].

That's what ConnectHub is: a multi-client LAN chat and file-sharing app written in plain C. The server uses POSIX threads to handle several clients at once, and the client is a small terminal program that uses `select()` to watch the keyboard and the socket at the same time. Nothing fancy, no GUI library, no external framework, just the standard C library and POSIX system calls (`socket`, `bind`, `listen`, `accept`, `pthread_create`, `select`, `read`, `write`).

Users can chat in public rooms, message each other privately, see who's online, and share files, all through one simple, custom, text-based protocol that's easy to read and debug. The point of the project is to actually use the tools taught in the course, threads, sockets, synchronization, I/O multiplexing, to build something that works end to end.

2. Problem Statement

Most chat and file-sharing apps we use every day (WhatsApp, Discord, etc.) need the internet and someone else's server. That's fine normally, but it's not great when you just want to chat and share files with people on the same local network, a lab, a classroom, an office, with no internet at all.

There's also a learning gap: ready-made chat libraries hide exactly the stuff a systems programming course is trying to teach, how sockets actually work, how threads share and protect data, how a protocol gets designed. Some specific problems this project is solving:

- No internet-free option: most chat tools simply won't work without an internet connection and outside servers.
- Black-box protocols: commercial apps hide their message format, so there's nothing to learn from how they actually talk over the wire.
- Weak concurrency in simple demos: a lot of beginner socket examples only handle one client at a time, which skips the hard (and useful) part, managing many clients safely with threads.
- No rooms or private chat: simple demos usually just broadcast to everyone, with no separate rooms or DMs.
- No file sharing: most teaching examples stop at text messages and never touch file transfer.

ConnectHub tackles all of this: a self-contained LAN chat and file-sharing tool, built from plain C and POSIX calls, with a protocol you can actually read and no hidden dependencies.

3. Objectives

3.1. Main Objective

To build a multi-client LAN chat and file-sharing app in plain C, a concurrent server plus a terminal client, using only standard POSIX tools (sockets, pthreads, select/poll), that lets people chat in public/private and share files in real time.

3.2. Specific Objectives

- Build a concurrent TCP server that handles many clients at once using threads, with mutexes protecting shared data (users, rooms).
- Design a simple, line-based text protocol for login, public/private messages, user listing, and file transfer.
- Build a terminal client that uses select()/poll() to handle keyboard input and network data at the same time, without blocking.
- Add file sharing between clients, routed through the server in small chunks over the same connection.
- Test everything, multi-client login, concurrent messaging, file transfer, and check the actual network traffic with tools like Wireshark/tcpdump.

4. Literature Review

4.1. Review of Related Theory

A few core ideas from the course underpin this whole project.

System calls are the bridge between a user program and the kernel, things like `open()`, `read()`, `write()`, `close()` give direct access to file descriptors, and in Unix, sockets are file descriptors too [3]. The Berkeley Sockets API (`socket`, `bind`, `listen`, `accept`, `send`, `recv`) is the standard way programs talk over TCP/IP [4]. TCP itself guarantees ordered, reliable delivery, which matters for chat and file transfer where messages need to arrive in order [5].

For handling many clients at once, servers usually spawn a thread (or process) per connection. POSIX threads let a server serve multiple sockets from one process, sharing memory like the user list, but shared memory means shared data needs protection, usually via mutexes, to avoid race conditions and deadlocks [6].

On the client side, `select()` (or `poll()`) lets a single-threaded program watch multiple file descriptors, like `stdin` and a socket, at once, and react to whichever is ready first. That's how a terminal chat client stays responsive without needing extra threads or a GUI event loop [4].

For the protocol itself, text-based line-delimited formats, like IRC, are simple, human-readable, and easy to debug compared to binary formats, which makes them a good fit for a small project like this [7].

4.2. Review of Related Works

Basic socket-programming tutorials (echo servers, single-client demos) teach the fundamentals but usually stop at one client, with no concurrency, rooms, or file transfer [8].

Multi-threaded chat demos go a step further, one thread per client, broadcasting to everyone, but usually only support a single global channel, with nothing like rooms or file sharing [6].

IRC itself is a good real-world example of a simple, readable, line-based protocol scaling to support channels and private messages, and it's a big influence on ConnectHub's own protocol [7].

Standard course exercises, echo servers, iterative vs. concurrent TCP servers, `select()`-based clients, give the basic building blocks (socket setup, accept loop, thread dispatch, non-blocking input) that this project pulls together into one complete app [4].

4.3. Gap Analysis

- Beginner socket examples usually handle just one client, with no real concurrency [8].
- Simple multi-threaded demos support only one broadcast channel, no rooms, no DMs [6].
- Few teaching examples add file transfer on top of chat.
- Terminal clients in course material usually block on one input source at a time, instead of using `select()/poll()` to handle both keyboard and network cleanly.
- Most examples pick either chat or file transfer, rarely both in one cohesive app.

ConnectHub tries to close these gaps: a proper concurrent server with rooms, a `select()`-based terminal client, an IRC-style protocol, and file sharing, all with nothing but plain C and POSIX calls, no GUI toolkit or external library.

5. Proposed Methodology

5.1. Project Approach

We're building this as a real, working product: a command-line chat and file-sharing app in C. Start with protocol design, then build the server, then the client, wire them together, and finish with testing and docs.

5.2. System Architecture & Design Modules

Two-tier setup: one server holds the truth (who's connected, which rooms exist, who's in them), and clients connect to it over TCP. Each client gets its own thread on the server (`pthread_create`) so nobody blocks anybody else. On the client side, `select()` watches both the keyboard and the socket at once, so there's no need for a second thread just to read input.

Modules:

- Server Core: sets up the socket (`socket`, `bind`, `listen`, `accept`) and spins up a thread per client.
- Room Management: tracks who's in which room, protected with mutexes.
- Login: username-based, rejects duplicates.
- Messaging: routes public room messages and private DMs between client threads.
- Presence: broadcasts who's online when people connect/disconnect.
- File Sharing: reads/writes files in chunks over the same connection, routed through the server.
- Logging: writes server events to a log file.
- Client I/O: a `select()` loop juggling `stdin` and the socket.
- Command Parser: turns things you type (`/join`, `/msg`, `/sendfile`) into protocol messages.

5.3. Technology Stack

- Language: C
- Networking: Berkeley sockets (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`) over TCP.
- Concurrency: `pthreads` for server threads, mutexes for shared state.
- I/O multiplexing: `select()` (or `poll()`) on the client.
- Nothing else: just the C standard library and POSIX calls, no GUI toolkit, no external framework.

- Build: a Makefile, output binaries in bin/.
- Protocol: our own line-based text format (TYPE|field1|field2|...), shared via a common header.
- Debugging tools: netstat/ss and tcpdump/Wireshark to check what's actually happening on the wire.

5.4. Application Protocol Design

Messages are plain newline-terminated strings like TYPE|field1|field2|field3, easy to read, easy to debug. Types include LOGIN, LOGIN_OK, LOGIN_FAIL, PUBLIC, PRIVATE, USERS, ROOMS, FILE_OFFER, FILE_CHUNK, FILE_DONE, and NOTIFY. Basically IRC's simplicity, shaped around ConnectHub's own features.

5.5. Development Methodology

We're using the Prototype Model: get a bare-bones version working first (login + public messages, like an echo server), then add rooms, DMs, presence, and file sharing one at a time, testing as we go before moving to the next piece.

5.6. Concurrency & Data Handling Strategy

Shared server state, the user list, room membership, is behind mutexes so concurrent client threads don't step on each other. Each client's thread cleans up its socket and resources properly on disconnect. File transfers just move in fixed-size chunks over the existing TCP stream, no extra library needed.

5.7. Testing & Validation Plan

- Basic checks: test protocol parsing and room/user logic as we build it.
- Integration testing: spin up the server and several clients together, check login, listing, messaging, and file transfer all work end to end.
- Concurrency/stress testing: hammer it with multiple clients at once to catch race conditions or deadlocks.
- Wire-level checks: use tcpdump/Wireshark to confirm messages are actually framed and delivered correctly.

5.8. Deployment Plan

Run the server (chatserver) on one machine, binding to a port (default 8080). Clients (chatclient) run on any machine on the same network, pointed at that host/port. No hosting, no cloud, no GUI dependencies, just run the binaries.

5.9. Evaluation Metrics

- Feature-complete: chat, rooms, DMs, presence, and file sharing all work and pass the test suite.
- Concurrency-safe: handles multiple clients at once without crashing, deadlocking, or racing.
- Handles bad input: doesn't crash on malformed protocol messages.
- Responsive: the client keeps up with both typing and incoming messages without lag.
- Stable: survives extended use, repeated connects, disconnects, messaging, file transfers.

6. Time Schedule

We're giving this 1 month, broken down in the Gantt chart below.

Activity	W1	W2	W3	W4
Protocol Design & Requirements Analysis				
Server Core & Concurrency Implementation				
Terminal Client Development (select-based I/O)				
Feature Integration (Rooms, Private Msg, Files)				
Testing & Packet-Level Verification				
Documentation & Report Writing				
Final Presentation				

References

- [1] R. Sommerville, Software Engineering, 10th ed. Harlow, England: Pearson, 2016.
- [2] B. A. Forouzan, Data Communications and Networking, 5th ed. New York: McGraw-Hill Education, 2017.
- [3] W. R. Stevens, Advanced Programming in the UNIX Environment, 3rd ed. Boston, MA: Addison-Wesley, 2013.
- [4] W. R. Stevens, UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Boston, MA: Addison-Wesley, 2003.
- [5] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Boston, MA: Pearson, 2021.
- [6] D. R. Butenhof, Programming with POSIX Threads. Boston, MA: Addison-Wesley, 1997.
- [7] J. Oikarinen and D. Reed, "Internet Relay Chat Protocol," Internet Engineering Task Force (IETF), RFC 1459, May 1993.
- [8] B. W. Kernighan and R. Pike, The UNIX Programming Environment, Latest ed. Englewood Cliffs, NJ: Prentice Hall, 1984.